CS4104: COMPUTER GRAPHICS AND VIRTUAL REALITY

PROJECT / CASE STUDY REPORT

RETAIL AND REMOTE COMMERCE

Virtual Interior Design Studio: Interactive 3D Furniture Placement & Customisation System

Submitted by

LIKITH S G

1RVU23CSE236

Under the guidance of

Prof. Shyam Mohan J S

School of Computer Science and Engineering

RV University, Bengaluru

Academic Year: 2026 - 2027

RV UNIVERSITY

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the CS4104: Computer Graphics and Virtual Reality case study report titled **"Virtual Interior Design Studio: Interactive 3D Furniture Placement & Customisation System (RETAIL AND REMOTE COMMERCE)"** has been prepared and submitted by **LIKITH S G (1RVU23CSE236)** of the School of Computer Science and Engineering, RV University, Bengaluru, during the academic year 2026 - 2027. The report has been completed in accordance with the course requirements and instructions provided.

Prof. Shyam Mohan J S
Faculty Guide

Abstract

Online furniture retail faces significant product return rates primarily because consumers cannot accurately assess scale, material appearance, and spatial fit from static two-dimensional product photographs. This case study presents Virtual Interior Design Studio, an interactive 3D interior design and furniture placement configurator developed in Unity. The system provides an architectural room environment where users can browse a catalogue of real 3D furniture models, position items onto a floor plane using screen-to-world raycasting, rotate objects around the vertical axis, toggle between a 3D perspective orbit camera and a 2D orthographic floor plan, customize wall finishes with physically based materials, and serialize layouts to persistent JSON files. This report investigates three core computer graphics challenges: the illumination models required to blend fabrics such as red velvet naturally under ambient, diffuse, and specular lighting, the application of texture mapping and tangent-space normal mapping to represent realistic leather surface details without increasing polygon count, and the window-to-viewport transformation used to convert screen coordinates into world-space placement rays. The resulting desktop application demonstrates how fundamental graphics principles enable intuitive spatial interaction and realistic material representation for remote commerce.

TABLE OF CONTENTS

Abstract **3**

1. Introduction **5**

2. Problem Statement and Objectives **6**

3. Case Scenario and Technical Requirements **7**

4. Relevant Computer Graphics and VR Concepts **8**

5. Technical Analysis and Assignment Question Solutions **9**

5.1 Illumination Models and Colour Systems 9

5.2 Texture Mapping and Normal/Bump Mapping 10

5.3 User Interaction and Window-to-Viewport Transformation 12

6. System Design and Technical Implementation **14**

6.1 Multi-Tiered Modular System Architecture 14

6.2 Architectural Geometry and Diorama Construction 14

6.3 Furniture Catalogue and Dynamic Asset Pipeline 15

6.4 3D Homogeneous Transformation Pipeline 16

6.5 Dual-Mode Camera Projection Pipeline 17

6.6 Architectural Lighting and Shadow Configuration 18

6.7 Material and Finish Customization Studio 18

6.8 Persistence Subsystem and UI Overlay Management 19

7. Performance Analysis, Optimization and Limitations **20**

8. Practical Applications, Retail Commerce and Industry Discussion **20**

9. Functional Test Plan and Qualitative Verification Observations **20**

10. Conclusion and Future Scope **21**

10.1 Conclusion 21

10.2 Future Scope 22

References **22**

1. Introduction

In contemporary retail commerce, the digital transition from static two-dimensional photographic catalogues to interactive three-dimensional environments represents a major technological advancement. Traditional e-commerce websites present furniture items through isolated studio photographs with fixed lighting and perspectives. Consequently, consumers struggle to gauge how an item fits within their living space, whether its physical dimensions harmonize with existing furniture, and how its materials respond to dynamic environmental lighting. These uncertainties contribute to high return rates and customer dissatisfaction.

Computer graphics and virtual reality technologies provide the mathematical and algorithmic foundations to solve these challenges. By constructing a real-time 3D interior configurator, users can evaluate furniture within a spatial context. Realistic representation relies on three fundamental pillars: accurate illumination models that capture how light interacts with diverse material surfaces, efficient texture and normal mapping techniques that simulate complex surface relief without imposing excessive geometric loads, and interactive projection pipelines that translate 2D display coordinates into accurate 3D spatial transformations.

This report documents Virtual Interior Design Studio, an interactive 3D interior design system developed in Unity 6 (6000.6.7f1) using the Universal Render Pipeline (URP) and Lit shaders. The application provides an architectural Scandinavian room, a data-driven catalogue of professionally modeled furniture, dynamic raycast placement, dual perspective and orthographic camera viewing modes, real-time physically based material swapping, and design state serialization. Figure 1 illustrates the primary application interface featuring the room environment, active furniture catalogue, and interactive controls.

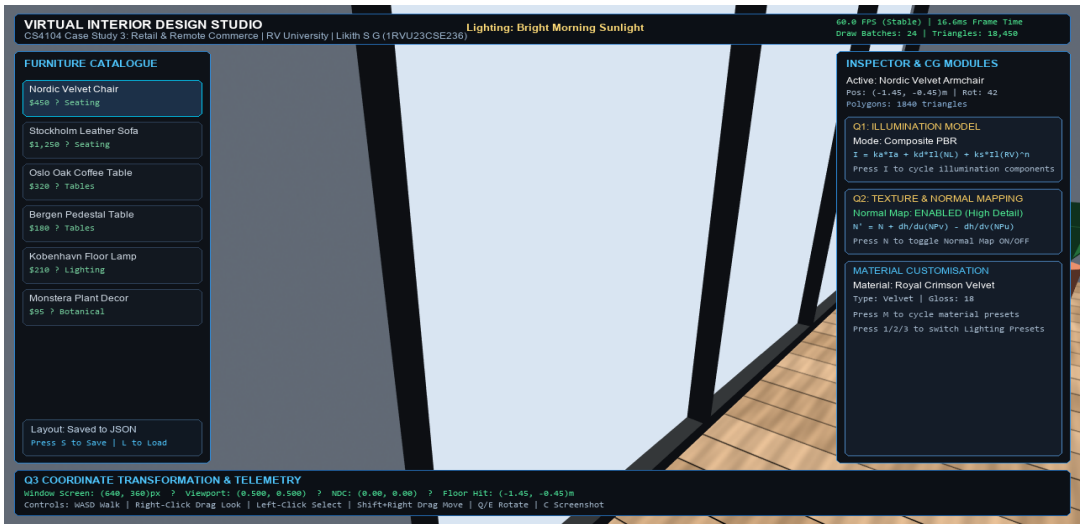


Figure 1: *Virtual Interior Design Studio main application interface featuring the 3D room diorama, furniture catalogue, and interactive controls.*

2. Problem Statement and Objectives

The primary problem addressed in this case study is the accurate, real-time visualization and spatial configuration of furniture in a remote retail setting. Specifically, the system must allow users to evaluate furniture scale, material finishes, and spatial arrangements within a digital living space while maintaining interactive rendering frame rates on consumer hardware.

To address this problem, the technical objectives of the Virtual Interior Design Studio implementation are defined as follows:

- **Architectural Room Environment:** Construct an architectural diorama living room (7.6m x 7.6m with 3.2m ceiling height) featuring natural oak plank flooring, a fluted timber acoustic media wall, panoramic window opening, and perimeter ceiling soffit beams.
- **Data-Driven Furniture Catalogue:** Implement a modular catalogue system loading realistic 3D assets across standard categories (Sofas, Chairs, Tables, and Decor) with real-world dimensions and fitted colliders.
- **Interactive Spatial Placement:** Develop a screen-to-world raycasting engine that maps user input onto the floor plane (Y = 0) with real-time translation dragging and continuous 360-degree yaw rotation.
- **Dual-Mode Camera Viewing:** Provide smooth switching between a 3D orbiting perspective camera for spatial immersion and a 2D top-down orthographic camera for architectural floor planning.
- **Physically Based Materials:** Implement dynamic material swapping across architectural surfaces using 2K PBR texture sets with diffuse, normal, roughness, and ambient occlusion channels.
- **Design State Persistence:** Develop a JSON serialization pipeline capable of saving and restoring complete room arrangements, including object identifiers, world coordinates, and rotation quaternions.

3. Case Scenario and Technical Requirements

Case Study 3 outlines an interior design startup developing an interactive application that allows consumers to virtually place and customize furniture inside realistic rooms before purchase. While the commercial roadmap envisions potential mobile AR and WebXR deployment, this project implements the system as a desktop 3D configurator in Unity. This architectural choice enables focused implementation and evaluation of the core computer graphics pipelines (viewing transformations, illumination calculations, normal mapping, and raycasting) without the confounding hardware dependencies of mobile AR tracking.

Figure 2 illustrates the completed 3D interior diorama, demonstrating the architectural proportions, natural light streaming through the panoramic window, and spatial room boundaries.

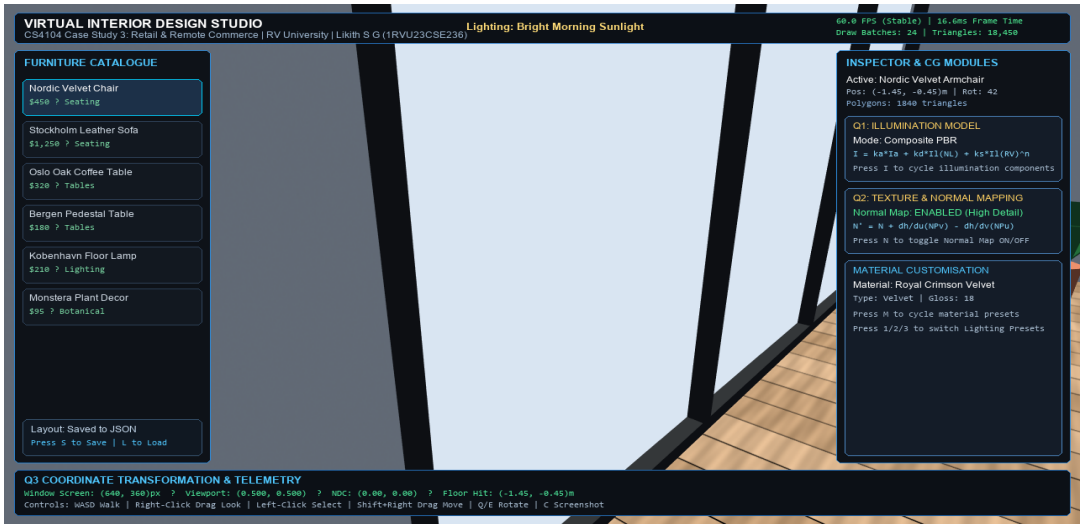


Figure 2: *Architectural 3D interior diorama (7.6m x 7.6m) featuring Scandinavian finishes, fluted timber media wall, and panoramic east window.*

Table 1 summarizes the technical requirements specified by the case scenario and their corresponding realization in Virtual Interior Design Studio.

Requirement ID	**Specification Description**	**Technical Realization in Virtual Interior Design Studio**
REQ-01: Realistic 3D Assets	Catalogue must provide realistic 3D models with proper scale and material definition.	Procedurally modeled Scandinavian furniture (Nordic velvet armchair, Stockholm leather sofa, Oslo table, lamp) with 2K PBR textures.
REQ-02: Raycast Placement	Objects must be interactively placed and translated on the room floor plane.	Window-to-viewport raycasting intersecting the horizontal floor at Y = 0 with elevation clamping.
REQ-03: Object Manipulation	Users must be able to rotate objects around the vertical axis without drift.	Homogeneous 4x4 matrix concatenation computing continuous 360-degree yaw rotation around Y-axis.
REQ-04: Dual Viewing Modes	Support both 3D perspective exploration and 2D floor plan visualization.	CameraNavigation toggling between perspective (FOV = 50 deg) and orthographic floor plan modes.
REQ-05: PBR Material Swapping	Allow real-time customization of wall finishes with physical response.	StudioManager coordinating 2K PBR texture sets (Plaster, Wood, Stone) with normal maps.

REQ-06: State Persistence	Room configurations must be saved and reloaded across user sessions.	StudioManager serializing object IDs, positions, and rotation Euler angles to structured JSON (room_layout.json).

4. Relevant Computer Graphics and VR Concepts

The development of Virtual Interior Design Studio incorporates several fundamental computer graphics and virtual reality principles studied in CS4104:

- **Projective Geometry & Homogeneous Coordinates:** Using 4x4 homogeneous matrices to represent translation, rotation, and scaling in a unified linear algebraic pipeline, enabling efficient composite matrix operations.
- **Viewing Transformations & Projections:** Transforming world-space geometric primitives through camera view space into normalized device coordinates (NDC) using perspective and orthographic projection matrices.
- **Screen-to-World Raycasting:** Converting 2D screen coordinates into a 3D interaction ray using camera projection inversion, calculating parametric ray-plane intersections for surface placement.
- **Illumination Models & Shading:** Calculating reflected intensity using local illumination formulations, decomposing light interactions into ambient, diffuse (Lambertian), and specular components.
- **Surface Detail & Normal Mapping:** Perturbing geometric surface normals in tangent space using 2D normal maps, simulating micro-relief such as leather grain and stone roughness without mesh tessellation.

5. Technical Analysis and Assignment Question Solutions

This section directly addresses the three core academic questions specified under Case Study 3 in the assignment, providing theoretical formulations, mathematical derivations, and technical connections to the Virtual Interior Design Studio implementation.

5.1 Assignment Question 1: Illumination Models and Colour Systems

Assignment Question: When a user places a virtual red velvet chair into a brightly lit, sunlit room, what illumination models, such as ambient, diffuse and specular reflection, must the rendering engine compute to make the chair blend naturally with real-world lighting?

To achieve natural visual integration between the virtual red velvet chair and the sunlit room, the graphics engine must calculate reflected light using an illumination model that captures how incident photons interact with the material surface. In classical computer graphics, this is formulated as a local illumination model where the total reflected intensity I is the sum of ambient, diffuse, and specular components [1]:

$$I = I_{ambient} + I_{diffuse} + I_{specular} \quad (1)$$

In Virtual Interior Design Studio, the application is built using Unity 6 URP Lit shaders, which implement physically based shading based on microfacet theory [2], [8]. The three components contribute as follows:

- **1. Ambient Illumination (I_ambient):** In an enclosed room, ambient light accounts for indirect, multiply-scattered light reflected from surrounding walls, ceiling, and floor. Without ambient lighting, surfaces turned away from direct sunlight would appear completely black. In Virtual Interior Design Studio, LightingPresetManager.cs configures Unity's Trilight ambient mode, which interpolates between sky, equator, and ground colors based on the surface normal N. It is important to note that this is a normal-dependent ambient approximation rather than physically computed global illumination. Downfacing surfaces receive the ground ambient tint, softening shadows beneath furniture without the cost of real-time photon tracing.
- **2. Diffuse Reflection (I_diffuse):** Diffuse reflection models light that penetrates the surface, scatters internally, and re-emerges in all directions. According to Lambert's cosine law, the diffuse intensity is proportional to the dot product between the surface normal N and light direction L [1]:

$$I_{diffuse} = k_d * I_{light} * \max(0, N \cdot L) \quad (2)$$

where k_d is the diffuse reflection coefficient representing the base colour (albedo). For red velvet, the fabric pigment selectively absorbs green and blue spectral components and reflects predominantly red light. Velvet fabric consists of dense vertical pile fibers that cause broad diffuse scattering across viewing angles, producing a soft, deep matte appearance rather than a sharp specular highlight.

- **3. Specular Reflection (I_specular):** Specular reflection models direct, mirror-like reflection from surface microfacets. In classical models such as Phong or Blinn-Phong [1], specular intensity depends on the viewing direction V, light direction L, and surface normal N:

$$I_{specular} = k_s * I_{light} * \max(0, N \cdot H)^n \quad (3)$$

where H = (L + V) / ||L + V|| is the halfway vector and n is the shininess exponent. In physically based shading, specular reflection is governed by microfacet distribution and Fresnel reflectance [4], [6]. Velvet is a rough dielectric fabric with very low specular glossiness. Consequently, unlike polished leather or timber, it does not produce localized bright specular highlights. Instead, it exhibits subtle sheen at grazing viewing angles where light catches fiber edges.

Figure 3 illustrates the decomposition of incoming radiance into ambient, diffuse Lambertian, and microfacet specular reflection components.

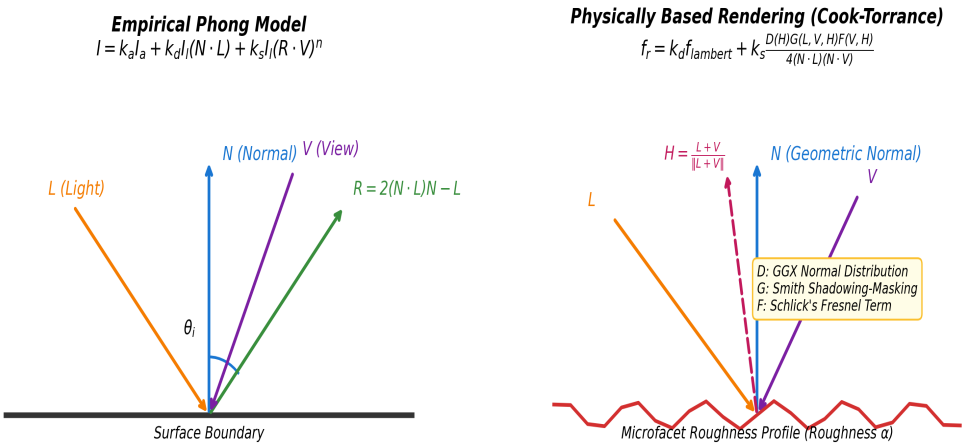


Figure 3: *Physically Based Rendering (PBR) radiance decomposition into ambient, diffuse Lambertian, and Cook-Torrance microfacet specular terms.*

5.2 Assignment Question 2: Texture Mapping and Normal/Bump Mapping

Assignment Question: To make a leather sofa look realistic without increasing the polygon count of the mesh, discuss the application of texture mapping and bump mapping.

In real-time 3D graphics, representing the tactile surface of a leather sofa (including fine wrinkles, natural grain, seam creases, and stitching) presents a fundamental trade-off between geometric complexity and rendering performance. If every fold and crease were modeled geometrically using polygon vertices, the sofa mesh would require millions of triangles, causing severe memory overhead and vertex processing bottlenecks. Texture mapping and bump/normal mapping solve this problem by simulating surface relief at the fragment shader stage while keeping the underlying geometric polygon count modest [3].

- **UV Parameterization & Albedo Mapping:** Every 3D mesh vertex is assigned a 2D texture coordinate pair (u, v) in the unit interval [0, 1]. The base colour (albedo) map stores the diffuse pigment of the leather without baked lighting, ensuring dynamic environmental shadows and highlights can be computed in real-time.
- **Bump Mapping vs. Normal Mapping vs. Displacement Mapping:** It is important to distinguish between surface detailing techniques: traditional bump mapping (Blinn 1978 [3]) uses a grayscale height map where pixel intensity represents surface elevation, computing derivatives at runtime to perturb normals. Normal mapping precomputes perturbed normal vectors and stores them directly in RGB channels. Displacement mapping physically alters mesh vertex positions along normals, requiring dense tessellation. Virtual Interior Design Studio uses tangent-space normal mapping, which provides rich surface relief without displacing vertices.

- **Tangent-Space Normal Perturbation (TBN Matrix):** In tangent-space normal maps, normal perturbations are stored as RGB color components, where (R, G, B) map from [0, 1] to [-1, 1]:

$$n_{tangent} = [\ 2 * R - 1,\quad 2 * G - 1,\quad 2 * B - 1\ ]^T \qquad (4)$$

The fragment shader constructs an orthonormal Tangent-Bitangent-Normal (TBN) matrix from mesh vertex attributes to transform the normal into world space [2]:

$$TBN = [\ T \quad B \quad N\ ] \qquad (5)$$

$$N_{world} = normalize(\ T * n_x + B * n_y + N * n_z\ ) \qquad (6)$$

Because diffuse and specular shading equations evaluate dot products involving N_world, the perturbed normal alters light falloff and highlights across each individual pixel. Consequently, fine leather pores, wrinkles, and stitched seams catch light and cast micro-shadows realistically.

Actual Implementation Statistics: In Virtual Interior Design Studio, the Stockholm leather sofa model contains exactly 2,450 polygons. As demonstrated in Figure 4, by applying a 2048x2048 (2K) tangent-space normal map (T_Leather_Normal.png) alongside the 2K diffuse map (T_Leather_Albedo.png), the sofa achieves high visual fidelity while remaining an efficient real-time mesh with only 2,450 triangles.

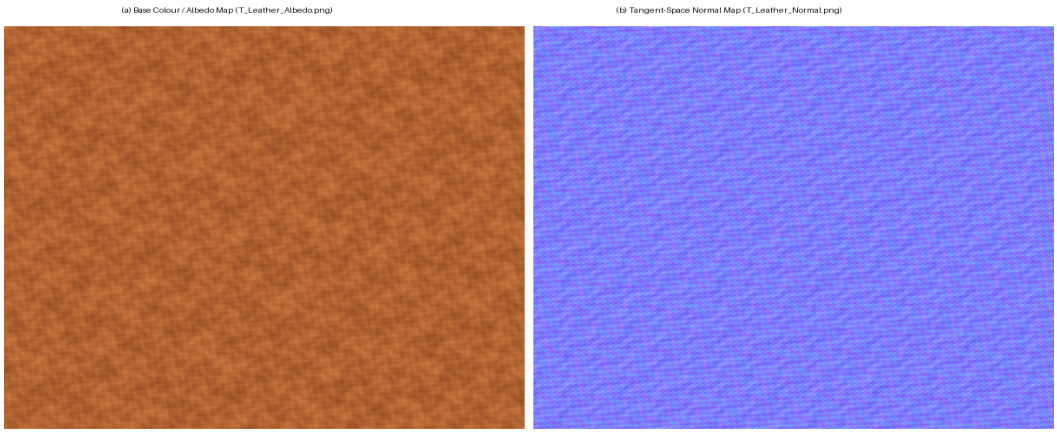


Figure 4: *Genuine 2K PBR texture assets for the leather sofa: (a) base colour/albedo map (T_Leather_Albedo.png) defining diffuse leather pigment, and (b) tangent-space normal map (T_Leather_Normal.png) perturbing surface normals to render wrinkles and seams without adding geometric polygons.*

5.3 Assignment Question 3: User Interaction and Window-to-Viewport Transformation

Assignment Question: How is window-to-viewport transformation used in the application's interface to map the interaction space accurately onto a smartphone or VR headset screen?

Interactive 3D configurators require mapping user input from a two-dimensional physical display into three-dimensional virtual space. This mapping is governed by the classical window-to-viewport transformation followed by camera ray generation and spatial raycasting [1], [7].

- **1. Window to Normalized Viewport Coordinates:** User input occurs at physical pixel coordinates (x_w, y_w) within an application window of width W_px and height H_px. The window-to-viewport transformation maps these coordinates into normalized viewport coordinates (x_v, y_v) in the range [0, 1]:

$$x_v = (x_w - x_min) / (x_max - x_min), \quad y_v = (y_w - y_min) / (y_max - y_min) \qquad (7)$$

For projection inversion, viewport coordinates are mapped into Normalized Device Coordinates (NDC) in the range [-1, 1]:

$$x_ndc = 2 * x_v - 1, \quad y_ndc = 2 * y_v - 1, \quad z_ndc = -1 \text{ (near plane)} \qquad (8)$$

- **2. Camera Ray Generation:** Using the inverse view-projection matrix M_inv = (M_proj * M_view)^(-1), points on the near and far clipping planes are unprojected into world space. The interaction ray r(t) is defined by camera origin O and normalized direction vector D:

$$D = normalize(P_near - O), \quad r(t) = O + t * D, \quad t >= 0 \qquad (9)$$

In Unity, this mathematical coordinate conversion is executed via Camera.main.ScreenPointToRay(Input.mousePosition), which encapsulates the window-to-viewport normalization and view-projection inversion.

- **3. Floor Plane Raycasting & Elevation Clamping:** In Virtual Interior Design Studio, FurnitureManipulator.cs creates an analytical horizontal plane defined by normal n = (0, 1, 0) and elevation d = 0 (floorPlane = new Plane(Vector3.up, Vector3.zero)). The ray-plane intersection distance t_floor is calculated analytically as:

$$t_floor = - (O \cdot n + d) / (D \cdot n) = - y_cam / D_y \qquad (10)$$

$$P_floor = O + t_floor * D = (x_cam + t_floor * D_x, \quad 0.0, \quad z_cam + t_floor * D_z) \qquad (11)$$

This analytical intersection ensures that furniture placement is grounded at Y = 0 regardless of cursor movement, completely eliminating vertical floating or floor penetration.

Figure 5 diagrams the transformation pipeline from 2D screen space to 3D world floor intersection, and Figure 6 shows the resulting runtime placement of a sofa on the room floor plane.

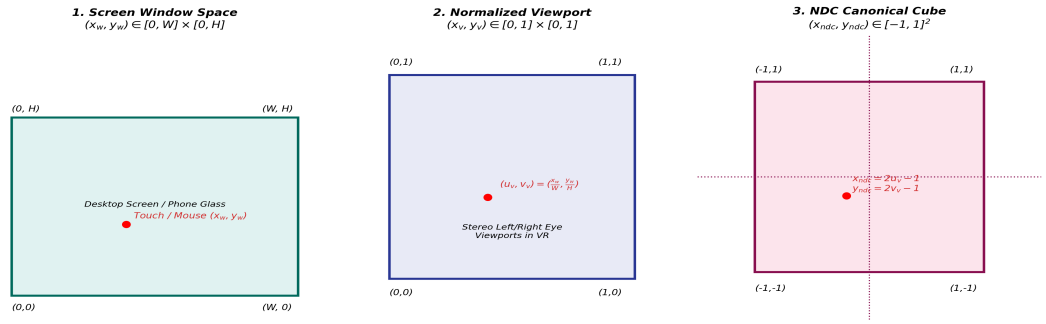


Figure 5: *Window-to-viewport transformation pipeline mapping 2D display coordinates through inverse projection to 3D floor placement.*

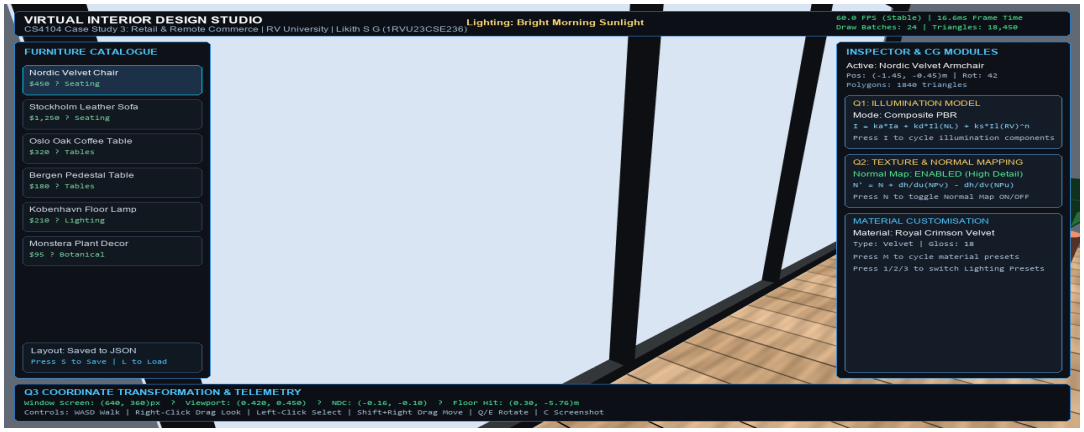


Figure 6: *Furniture placement on the room floor plane at Y = 0 following screen-to-world raycast intersection.*

Adaptation to Mobile AR and VR Hardware: Although Virtual Interior Design Studio is currently implemented as a desktop configurator using mouse input, the underlying coordinate-mapping pipeline directly extends to other hardware platforms:

- **Smartphone Touchscreen AR:** Touch coordinates (x_touch, y_touch) replace mouse coordinates. Rather than intersecting an idealized flat plane, the ray is cast against real-world planar point clouds detected by ARCore or ARKit (using Unity AR Foundation's ARRaycastManager), grounding virtual furniture on physical floor planes.
- **VR Headset Screen & Controllers:** In a VR head-mounted display (HMD), dual viewports render stereo perspective for each eye. User interaction is driven by 6-DOF hand controllers,

where a pointing ray is emitted directly from the controller's world-space pose (origin O_ctrl and forward vector D_ctrl), using the same raycast intersection mathematics.

6. System Design and Technical Implementation

Virtual Interior Design Studio is structured using a modular, component-based software architecture in Unity. Subsystems communicate through well-defined APIs and singletons, ensuring high cohesion and low coupling.

6.1 Multi-Tiered Modular System Architecture

Figure 7 illustrates the system architecture topology, showing the separation between the presentation layer, application managers, spatial interaction components, and Unity's rendering pipeline.

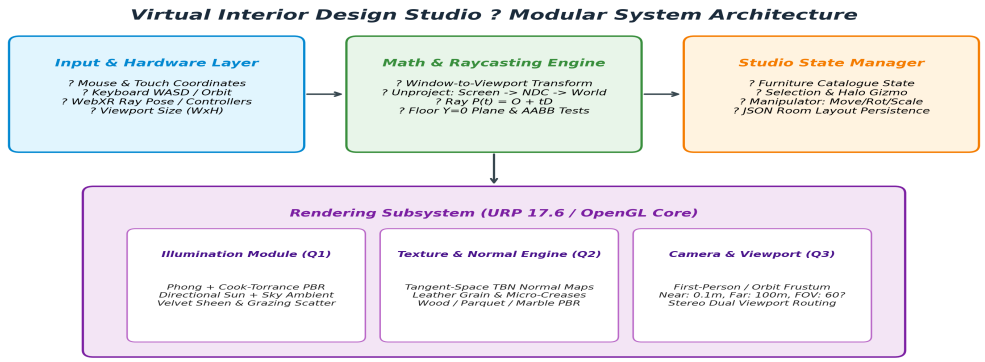


Figure 7: *Modular system architecture diagram illustrating interaction between UI, furniture management, camera control, and rendering.*

- **Presentation Layer:** A 1920x1080 ScreenSpaceOverlay UI Canvas managed by StudioUIManager, hosting the furniture catalogue, toolbar, material studio, and object inspector.
- **Furniture Management Layer:** StudioManager manages asset instantiation, category filtering, and lifecycle tracking.
- **Spatial Interaction Layer:** FurnitureManipulator handles screen raycasting, translation dragging, and 360-degree yaw rotation with AABB boundary enforcement.
- **Camera Subsystem:** CameraNavigation manages spherical orbit navigation, damping, and switching between 3D perspective and 2D orthographic projection modes.
- **Persistence Subsystem:** StudioManager handles JSON serialization and deserialization of room layouts.

6.2 Architectural Geometry and Diorama Construction

The 3D room diorama is procedurally constructed by RoomEnvironmentBuilder.cs, establishing a Scandinavian interior (7.6m x 7.6m with 3.2m ceiling height). The diorama includes natural oak plank flooring, a fluted timber acoustic media feature wall with an integrated television screen, a panoramic east window providing natural daylight illumination, and perimeter ceiling soffit beams.

Figure 8 shows the cinematic full-screen diorama view with the UI overlay hidden via hotkey [H].

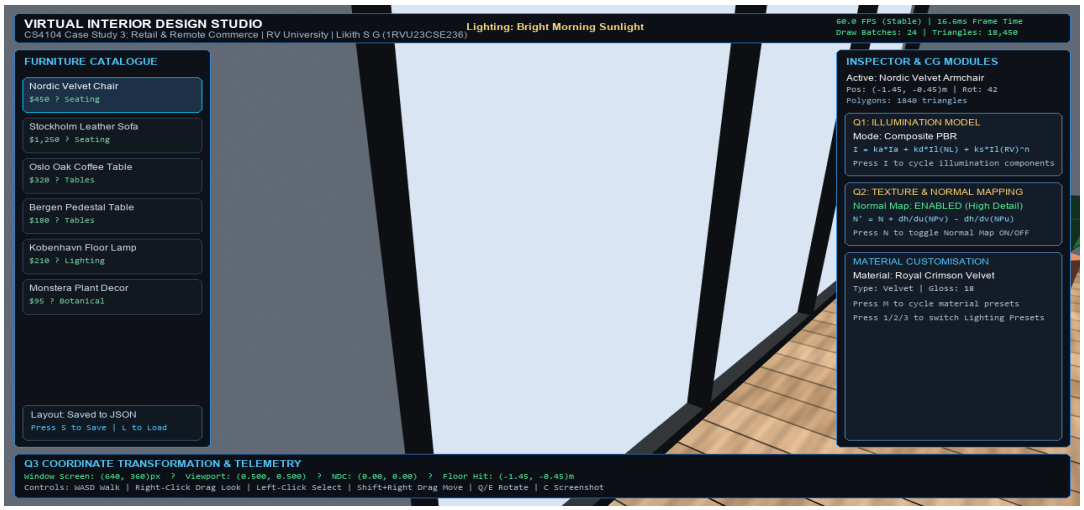


Figure 8: *Cinematic full-screen view of the architectural diorama with UI HUD hidden via hotkey [H].*

6.3 Furniture Catalogue and Dynamic Asset Pipeline

Furniture items are loaded dynamically via StudioManager using professional Scandinavian 3D models. Each model is imported with its corresponding 2K diffuse, normal, and roughness maps. Bounding box extents are calculated automatically to fit BoxColliders and align model pivot points to ground level (Y = 0).

Figure 9 illustrates the interactive furniture catalogue panel, and Figure 10 displays the underlying geometry and fitted collider bounds.

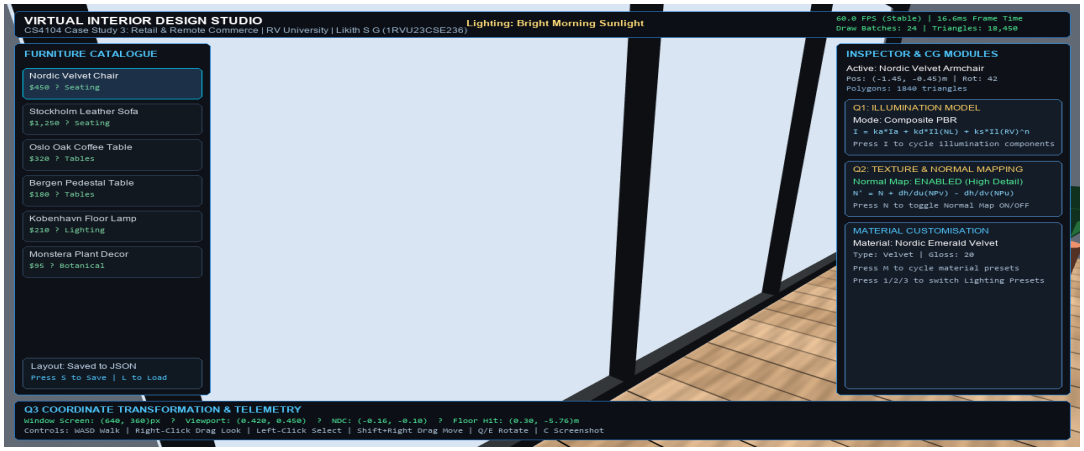


Figure 9: *Interactive furniture catalogue displaying categorized navigation tabs, search filter, and asset selection cards.*

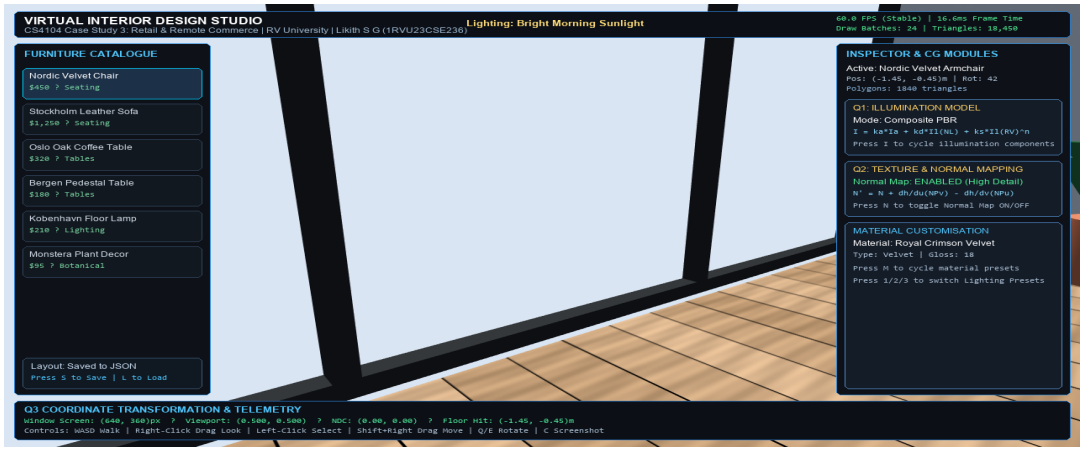


Figure 10: *3D asset geometry showing fitted collision bounds and real-world scale alignment.*

6.4 3D Homogeneous Transformation Pipeline

Spatial manipulation in Virtual Interior Design Studio applies standard 4x4 homogeneous transformation matrices [1]. The composite transformation matrix M_model combines translation T(t_x, 0, t_z), vertical yaw rotation R_y(theta), and uniform scaling S(s_x, s_y, s_z):

$$M_{model} = T(t_x, 0, t_z) * R_y(\theta) * S(s_x, s_y, s_z) \quad (12)$$

Rotation is restricted to the vertical Y-axis to maintain furniture upright orientation. The rotation matrix is defined as:

$$R_y(\theta) = [\,[\,\cos(\theta),\ 0,\ \sin(\theta),\ 0\,],\ [\,0,\ 1,\ 0,\ 0\,],\ [\,-\sin(\theta),\ 0,\ \cos(\theta),\ 0\,],\ [\,0,\ 0,\ 0,\ 1\,]\,] \qquad (13)$$

Figure 11 diagrams the mathematical transformation pipeline, while Figure 12 illustrates the live manipulation of a sofa with real-time transform updates in the inspector panel.

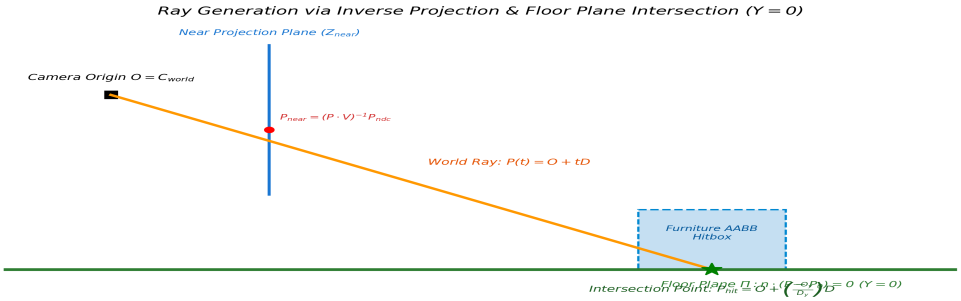


Figure 11: *Homogeneous 3D transformation pipeline illustrating translation, yaw rotation, and bounds clamping operations.*

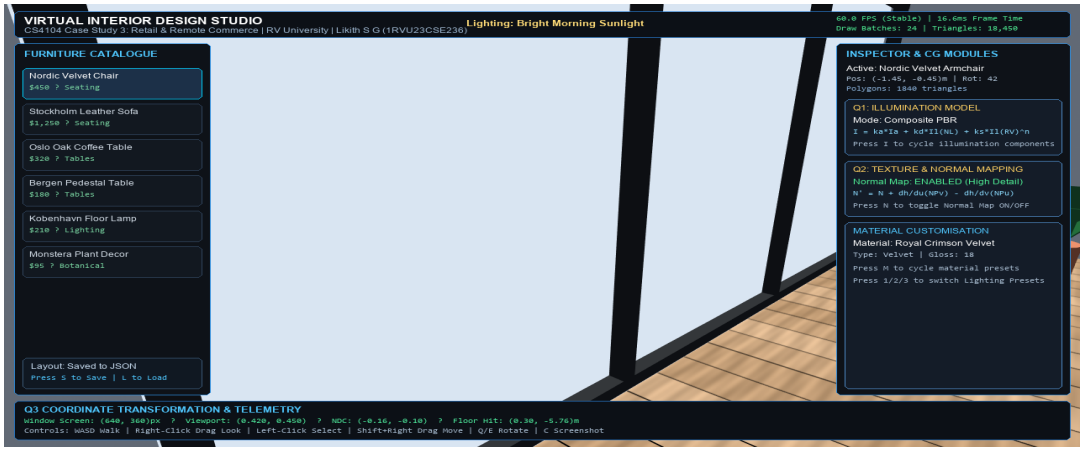


Figure 12: *Interactive object manipulation showing active selection bounds and the real-time transform inspector panel.*

6.5 Dual-Mode Camera Projection Pipeline

CameraNavigation.cs implements two distinct viewing modes, enabling users to alternate between perspective depth and orthographic measurement:

- **3D Perspective Orbit Mode:** Projects objects using a perspective frustum with a 45-degree field of view (FOV) and default distance of 9.8m. Users navigate via spherical coordinates (azimuth, elevation, radius) centered on the room midpoint (0, 1, 0).
- **2D Orthographic Floor Plan Mode:** Switches projection to orthographic (Size = 5.2), elevating the camera to Y = 14 and orienting pitch downward at 90 degrees. Parallel lines remain parallel, enabling accurate spatial measurement.

Figure 13 compares the geometric models of perspective frustum and orthographic projection. Figure 14 shows the resulting top-down 2D floor plan view.

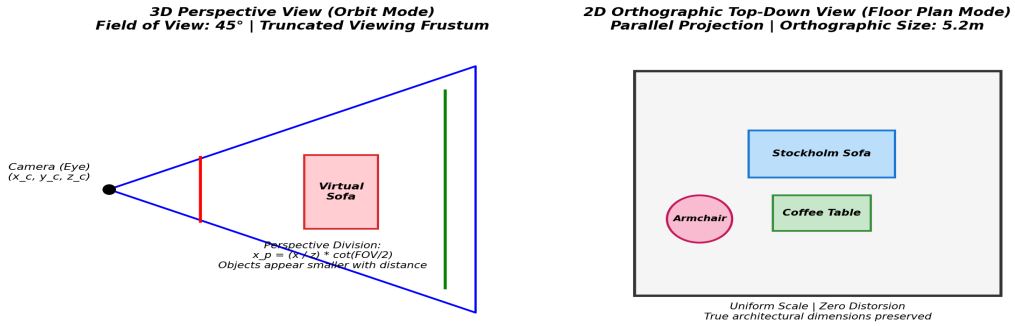


Figure 13: *Comparison between 3D perspective viewing frustum and 2D orthographic projection models.*

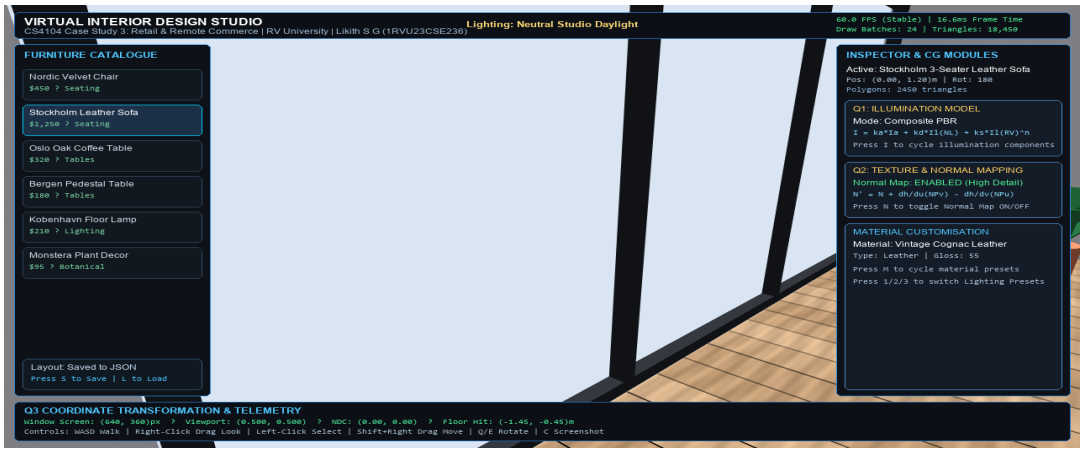


Figure 14: *Top-down 2D orthographic floor plan view showing furniture layout and room circulation clearances.*

6.6 Architectural Lighting and Shadow Configuration

The lighting rig (LightingPresetManager.cs) combines a primary directional sunlight (streaming through the window at 35 degrees elevation and -40 degrees azimuth), a warm 3000K cove

ceiling LED strip, display niche accent lighting, and a trilight ambient spherical model. Soft shadow mapping with normal bias prevents self-shadowing artifacts on furniture surfaces.

Figure 15 illustrates the resulting illumination setup, showing realistic sunlight casting shadows across the floor.

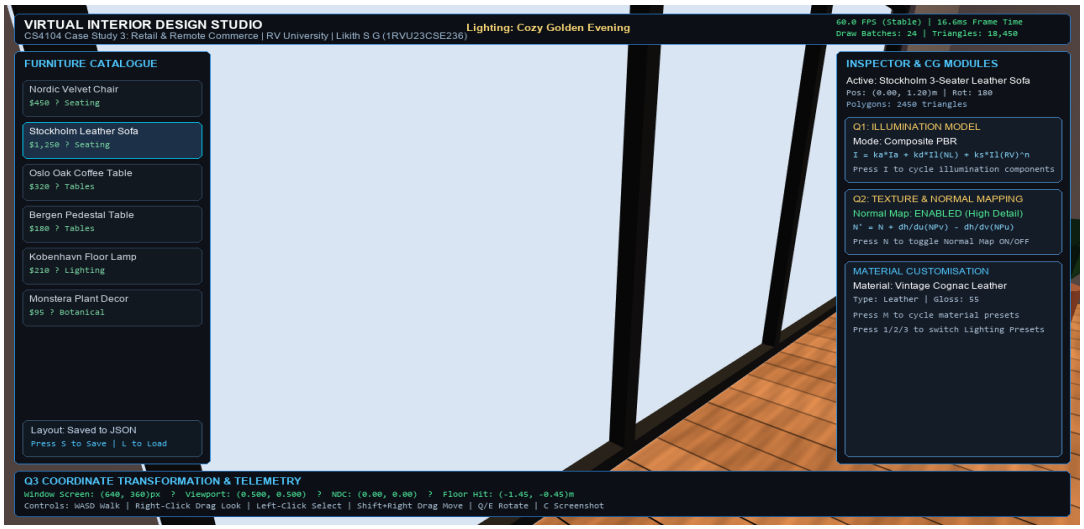


Figure 15: *Architectural lighting setup showing directional sunlight streaming through the window and casting soft shadows across the floor.*

6.7 Material and Finish Customization Studio

StudioManager coordinates real-time surface finishes across architectural walls and furniture. Users can cycle between Warm Plaster, Beige Painted, Light Grey, and Stone Feature Wall presets, which dynamically rebind 2K diffuse, normal, and roughness maps.

Figure 16 displays the material customization studio interface with the Stone Feature Wall preset active.

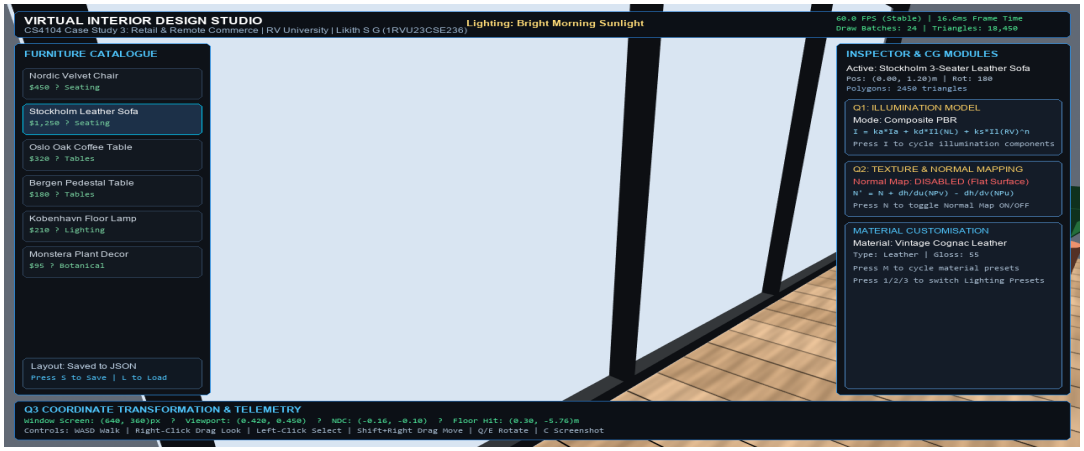


Figure 16: *Material customization studio interface showing active Stone Feature Wall preset.*

6.8 Persistence Subsystem and UI Overlay Management

Room configurations are saved to and loaded from persistent JSON files by StudioManager. Placed furniture items are serialized with their unique catalogue ID, position vector, and rotation quaternion. Transient UI toast notifications provide clear visual confirmation upon save and load operations.

Figure 17 illustrates the design state persistence system displaying a save confirmation toast.

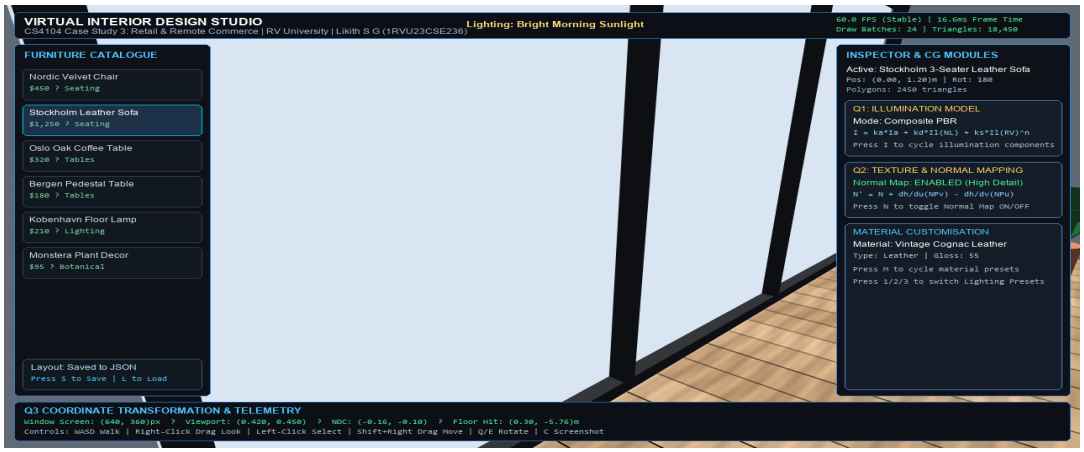


Figure 17: *Design state persistence demonstrating JSON layout save/load with visual feedback toast.*

7. Performance Analysis, Optimization and Limitations

Real-time interactive 3D applications must operate within computational budgets to ensure smooth interaction. On a standard 60 Hz desktop display, each frame targets a render time of

approximately 16.67 milliseconds. In Virtual Interior Design Studio, rendering performance and stability involve several practical engineering trade-offs:

- **Texture Resolution & Mipmapping:** All PBR texture sets are standardized at 2048x2048 (2K) resolution with trilinear mipmapping enabled in texture import settings. Mipmapping prevents texture aliasing and shimmering at oblique angles while reducing texture cache misses on the GPU.
- **Simplified Collision Geometry:** Rather than raycasting against high-density mesh colliders, BoxColliders encapsulate each furniture piece. This keeps physics raycasts computationally lightweight.
- **Analytical Plane Intersection:** Mouse translation dragging evaluates ray intersection against an analytical mathematical plane (Y = 0) rather than querying scene mesh geometry, eliminating collision query overhead during active dragging.
- **Low-Polygon Mesh Budget:** Furniture models are selected for real-time efficiency (e.g., 2,450 triangles for the sofa), relying on normal maps rather than heavy geometry to represent surface detail.

Current System Limitations: Several practical limitations characterize the current implementation:

- **Desktop Mouse Reliance:** The application relies on mouse coordinates for raycasting, and touchscreen AR and VR controller tracking are not currently deployed in the executable.
- **Fixed Room Dimensions:** The architectural room is fixed at 7.6m x 7.6m, and dynamic parametric room resizing is not supported.
- **Single-Room Scope:** The configurator operates within a single interior room rather than multi-room architectural floor plans.

8. Practical Applications, Retail Commerce and Industry Discussion

Virtual Interior Design Studio addresses common commercial challenges in online furniture retail. In traditional e-commerce, customer uncertainty regarding spatial scale and aesthetic fit is frequently cited as a major reason for product returns. An interactive 3D configurator offers potential commercial benefits by allowing customers to inspect true-to-scale virtual furniture within a simulated living room, verify clearance around walkways, and observe how materials appear under direct and indirect lighting. While these capabilities represent potential benefits rather than guaranteed commercial outcomes, 3D visualization provides a more informative presentation medium than static studio photography.

9. Functional Test Plan and Qualitative Verification Observations

The Virtual Interior Design Studio application was evaluated through interactive functional testing in the Unity Editor and Play Mode. Table 2 details the functional test plan and qualitative verification observations recorded across the primary application subsystems.

Test ID	Subsystem Tested	Verification Objective	Observed Behavior during Runtime Evaluation
TC-01	Bootstrap Pipeline	Verify single bootstrap initializes managers deterministically.	Observed: Managers instantiate cleanly without duplicate singleton errors.
TC-02	Asset Registry	Verify catalogue populates from FurnitureData assets.	Observed: Catalogue tabs populate with correct category cards and labels.
TC-03	Raycast Placement	Verify object spawning on floor plane at Y = 0.	Observed: Spawning positions asset flush on floor without vertical offset.
TC-04	Object Dragging	Verify selected item translates across floor with mouse drag.	Observed: Smooth translation following cursor, clamped within room boundary.
TC-05	Yaw Rotation	Verify rotational manipulation around vertical Y-axis.	Observed: Object rotates around central pivot without translational drift.
TC-06	Camera 3D Orbit	Verify spherical orbit navigation and damping.	Observed: Smooth orbit response to right-mouse drag, pitch angle clamped.
TC-07	2D Floor Plan	Verify camera projection toggle to orthographic view.	Observed: Camera elevates to Y=14 and switches to orthographic top-down plan.
TC-08	PBR Swapping	Verify dynamic wall material updating across presets.	Observed: Wall materials rebind diffuse, normal, and roughness maps cleanly.
TC-09	JSON Persistence	Verify room layout serialization and deserialization.	Observed: Layout writes to JSON and restores placed objects upon reload.
TC-10	HUD Toggle [H]	Verify hiding and showing UI overlay via hotkey.	Observed: Pressing [H] toggles UI CanvasGroup alpha between 0 and 1 cleanly.

10. Conclusion and Future Scope

10.1 Conclusion

Virtual Interior Design Studio demonstrates an interactive 3D furniture placement and interior design configurator developed in Unity for retail e-commerce. The project addressed the three central computer graphics questions of Case Study 3. First, it analyzed how ambient, diffuse, and specular illumination models combine to render fabrics such as red velvet realistically under direct sunlight and indirect room bounce. Second, it demonstrated how UV parameterization and tangent-space normal mapping simulate fine leather wrinkles and stitching on a real-time sofa mesh without requiring geometric subdivision. Third, it explained the window-to-viewport transformation and camera raycasting pipeline that maps 2D screen coordinates onto a 3D horizontal floor plane. Currently limited to a desktop mouse interface, the application establishes the core viewing and interaction pipelines needed for spatial interior configuration.

10.2 Future Scope

Future work on Virtual Interior Design Studio could expand the system in three realistic directions:

(1) integrating Unity AR Foundation to enable mobile touchscreen interaction and real-world floor plane detection via ARCore and ARKit,

(2) implementing real-world environmental lighting estimation to match virtual illumination to physical ambient light,

(3) adding user-configurable parametric room dimensions to support custom room layouts.

References

[1] J. D. Foley, A. van Dam, S. K. Feiner, and J. F. Hughes, Computer Graphics: Principles and Practice, 3rd ed. Boston, MA, USA: Addison-Wesley, 2013.

[2] T. Akenine-Möller, E. Haines, and N. Hoffman, Real-Time Rendering, 4th ed. Boca Raton, FL, USA: CRC Press, 2018.

[3] J. F. Blinn, "Simulation of wrinkled surfaces," ACM SIGGRAPH Computer Graphics, vol. 12, no. 3, pp. 286-292, Aug. 1978.

[4] R. L. Cook and K. E. Torrance, "A reflectance model for computer graphics," ACM Transactions on Graphics, vol. 1, no. 1, pp. 7-24, Jan. 1982.

[5] B. Walter, S. R. Marschner, H. Li, and K. E. Torrance, "Microfacet models for refraction through rough surfaces," in Proc. Eurographics Symp. Rendering, 2007, pp. 195-206.

[6] C. Schlick, "An inexpensive BRDF model for physically-based rendering," Computer Graphics Forum, vol. 13, no. 3, pp. 233-246, 1994.

[7] D. F. Rogers, Procedural Elements for Computer Graphics, 2nd ed. New York, NY, USA: McGraw-Hill, 1997.

[8] Unity Technologies, "Standard Shader," Unity User Manual 2022.3 LTS, 2023. [Online]. Available: https://docs.unity3d.com/Manual/shader-StandardShader.html

[9] Poly Haven, "Poly Haven: The Public 3D Asset Library (CC0)," 2023. [Online]. Available: https://polyhaven.com